



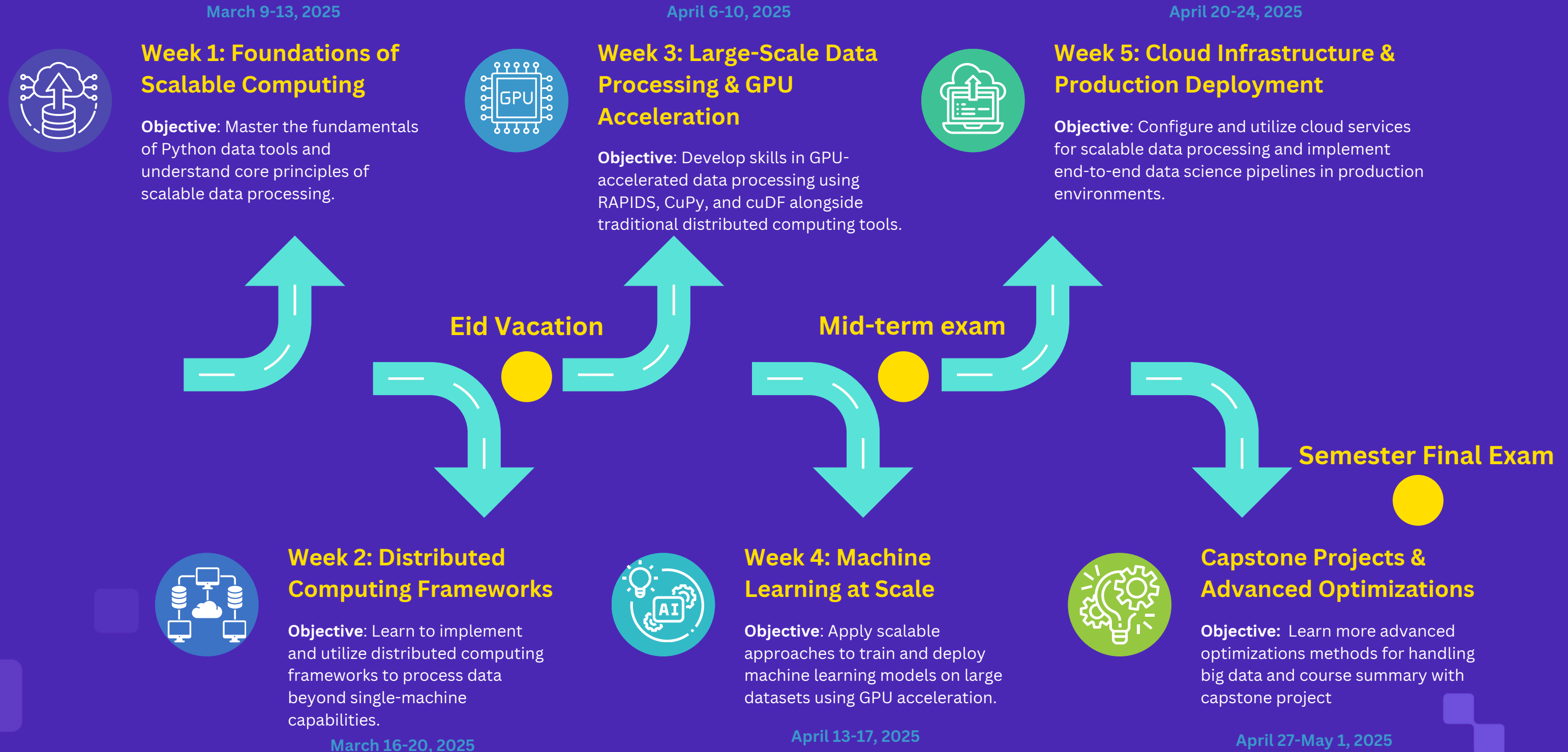
AI Diploma



Scalable Data Science with Python

Semester One | Course Three

SCALABLE DATA SCIENCE WITH PYTHON





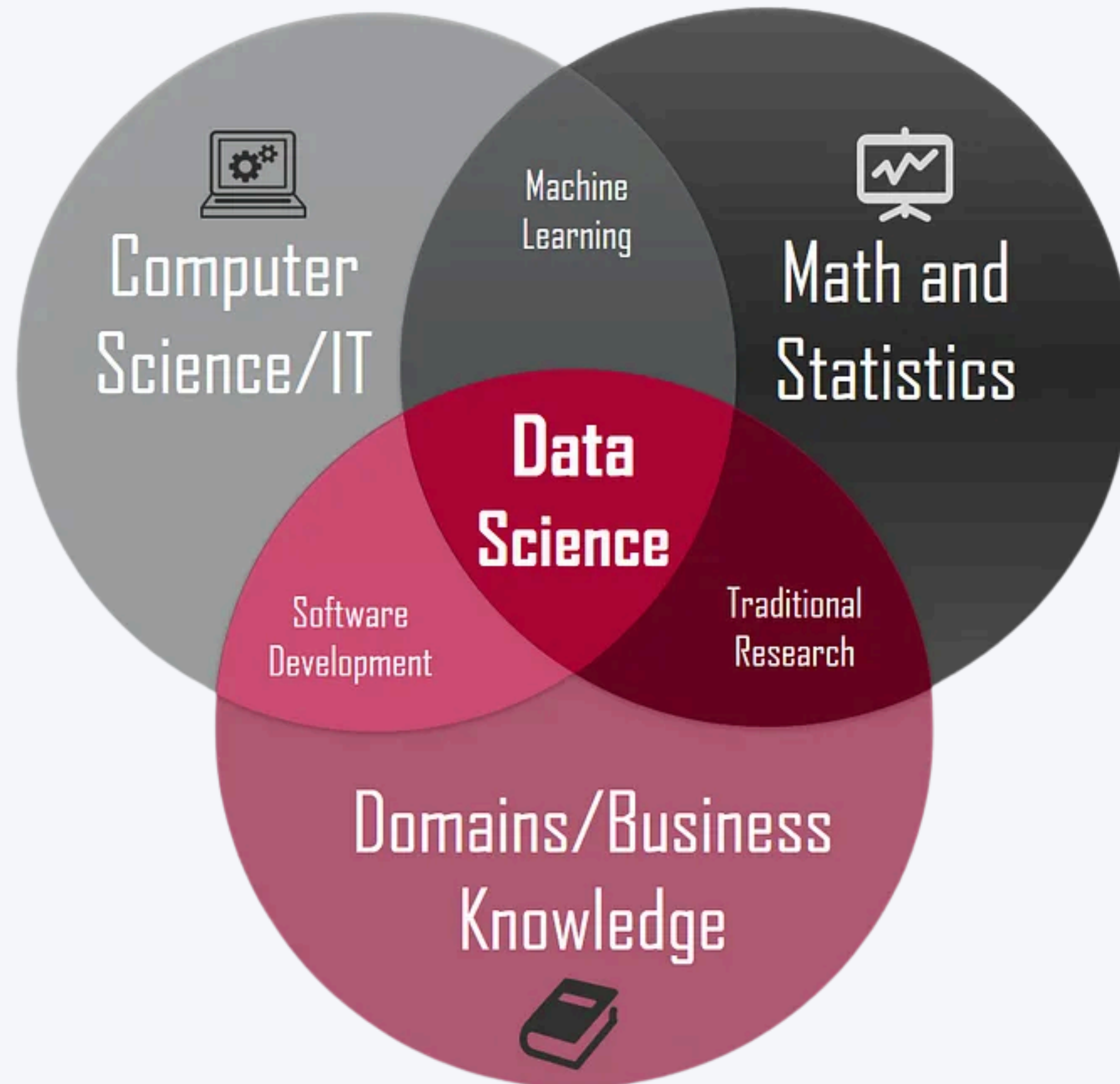
- Scalable Data Science with Python
Semester **One** | Course **Three**

Unit 1 : Foundations of Scalable Computing

Introduction to Data Science

Introduction to Data Science

What is Data Science?



What is Data Science?

Data Science is a combination of multiple disciplines that uses statistics, data analysis, and machine learning to analyze data and to extract knowledge and insights from it.

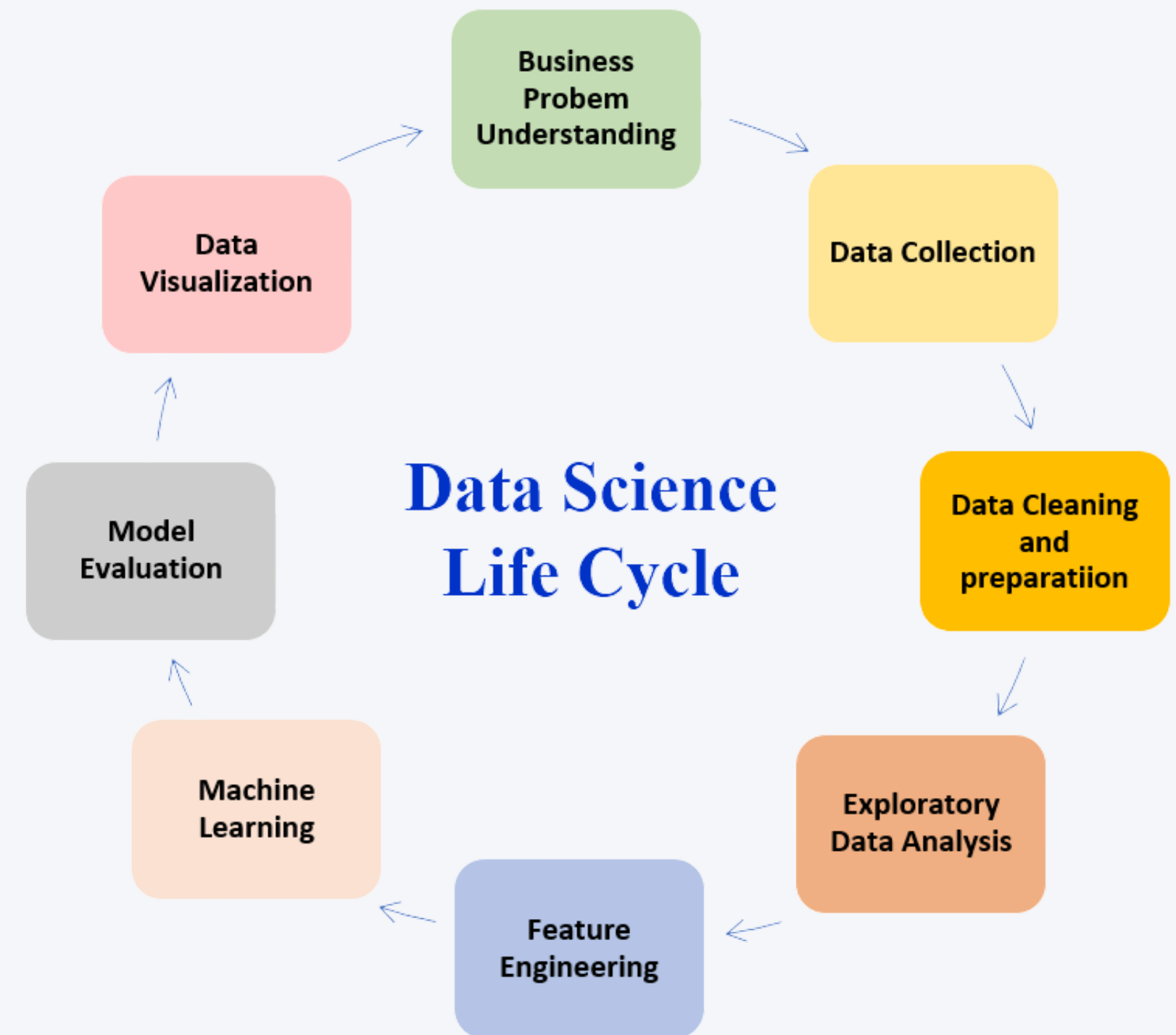
Key Components:

- Statistics and Mathematics
- Domain Expertise
- Programming and Database Knowledge
- Communication and Visualization Skills

The Data Science Lifecycle

Data Science Lifecycle is a step-by-step demonstration of how machine learning and other analytical methods are used to generate insights and predictions from data to achieve a business goal.

- **Business Understanding:** Define objectives and requirements
- **Data Acquisition:** Collect raw data from various sources
- **Data Preparation:** Clean, transform, and prepare data for analysis
- **Exploratory Data Analysis:** Understand patterns, and relationships
- **Modeling:** Develop predictive or descriptive models
- **Evaluation:** Assess model performance and validate results
- **Deployment:** Implement solutions in production environments
- **Monitoring:** Track performance and maintain systems



Data - Terms & Concepts

Data Acquisition

- The process of collecting and gathering raw data from various sources.
- This includes data capture from sensors, web scraping, database queries, API calls, or surveys...
- Data acquisition focuses on obtaining the raw materials needed for analysis.

Data Analysis

- The process of inspecting, cleaning, transforming, and modeling data to discover useful information, draw conclusions, and support decision-making.
- Data analysis typically follows a structured approach to examine what happened (**descriptive**), why it happened (**diagnostic**), what might happen (**predictive**), and what actions to take (**prescriptive**).

Data Mining

- A specific subset of data analysis that focuses on discovering patterns, anomalies, correlations, and dependencies in large datasets using automated or semi-automated techniques.
- Data mining applies algorithms specifically designed to extract hidden patterns that might not be apparent through standard analysis techniques.

Data - Terms & Concepts

Data Processing

- The conversion of raw data into a more useful format.
- This includes cleaning, normalization, transformation, and aggregation to prepare data for analysis.

Data Wrangling (or Data Munging)

- The process of cleaning, structuring, and enriching raw data into a desired format for better decision making.
- This addresses inconsistencies, missing values, and formatting issues.

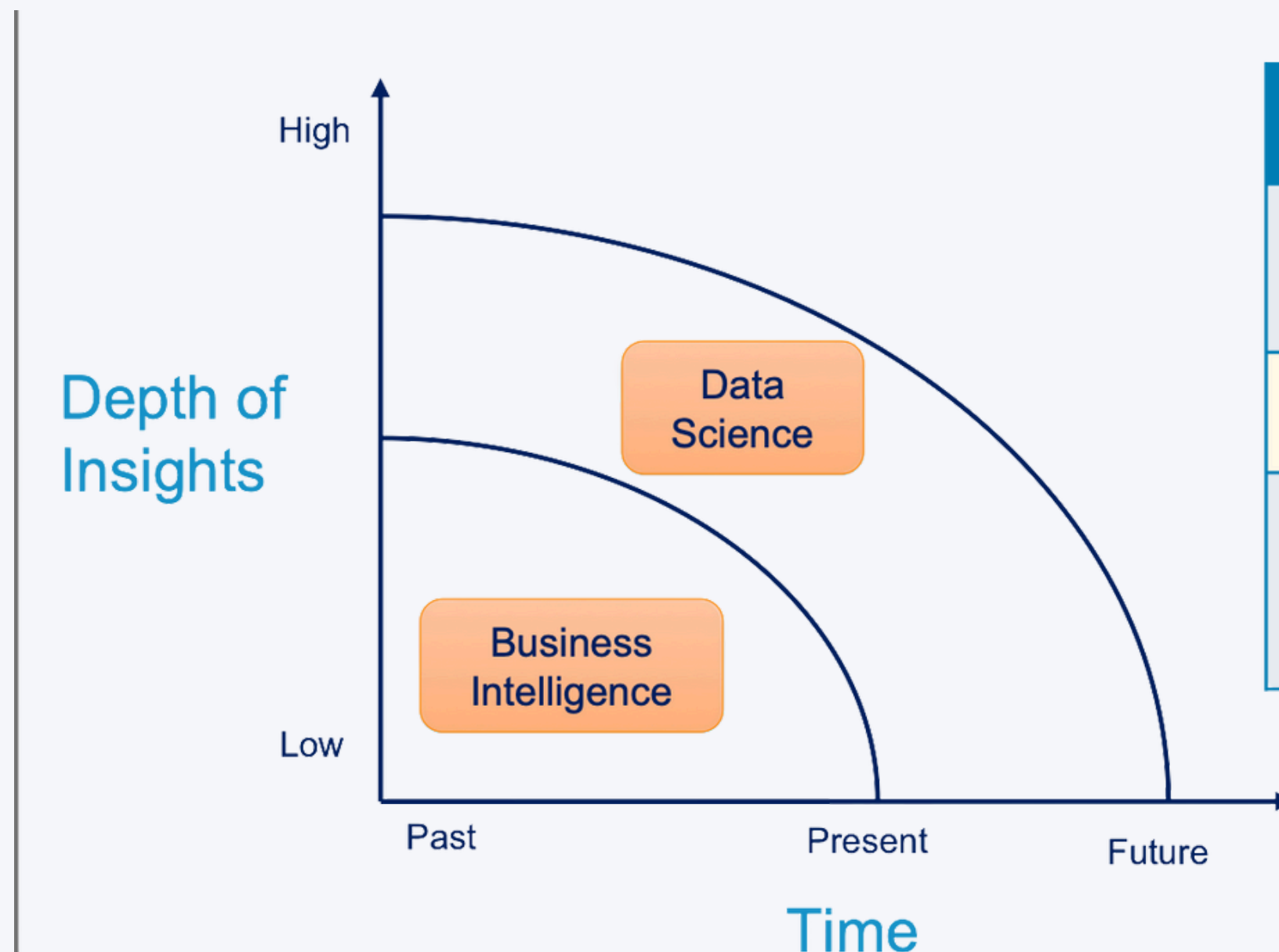
Data Exploration

- The initial investigation of data to discover patterns, spot anomalies, test hypotheses, and check assumptions using summary statistics and visualizations.

Data Science vs. Business Intelligence

Business Intelligence

- The strategies and technologies used for data analysis of business information, focusing on historical data to provide insights into past performance and guide business planning.



	Business Intelligence	Data Science
Techniques	Dashboards, alerts, queries	Optimization, predictive modelling, forecasting
Data Types	Structured, data warehouses	Any kind, often unstructured
Common questions	What happened...? How much did...? When did...?	What if...? What will...? How can we...?

What are Data Scientists?

Not computer scientists

- But should know about databases, data structures, algorithms, etc.

Not mathematicians

- But should know about optimization, stochastic, etc.

Not statisticians

- But should know about regression, statistical tests, etc.

Not domain experts

- But must work together with them

Introduction to Data Science



The Scale Challenge

Billions of sensors have been collecting data for decades

DNA sequencing lab, where massive amounts of genetic data are generated from sequencing machines.



The Scale Challenge

Billions of sensors have been collecting data for decades

large-scale radio telescope array used for astronomical observations, collecting vast amounts of raw radio frequency data from space

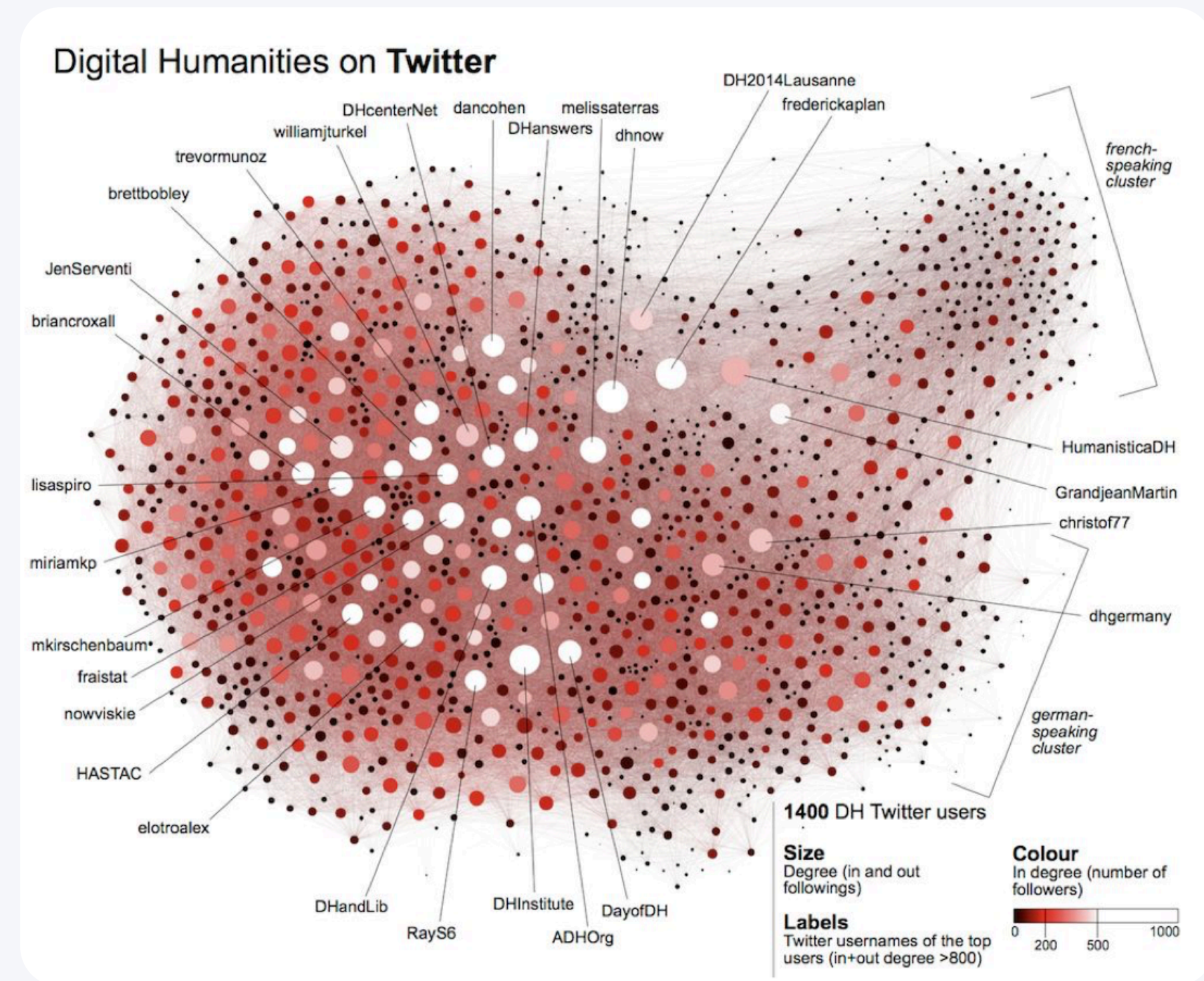


Introduction to Data Science

The Scale Challenge

Billions of sensors have been collecting data for decades

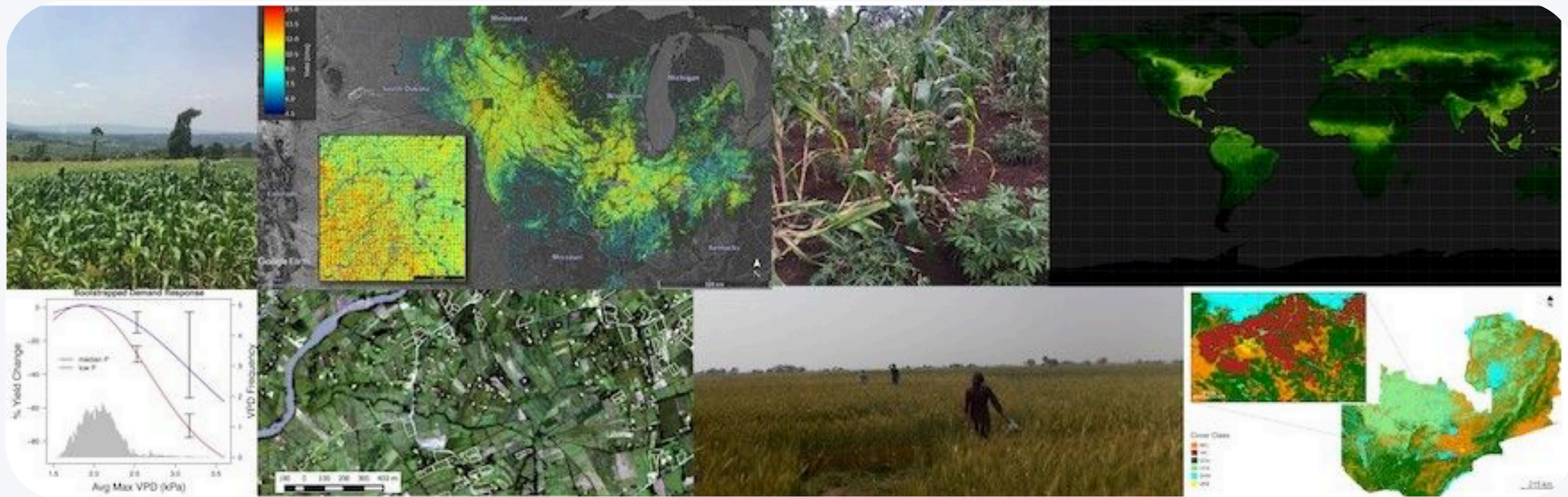
Social network graph, Mapping connections and interactions on platforms like Twitter, Handling billions of interactions.



Introduction to Data Science

The Scale Challenge

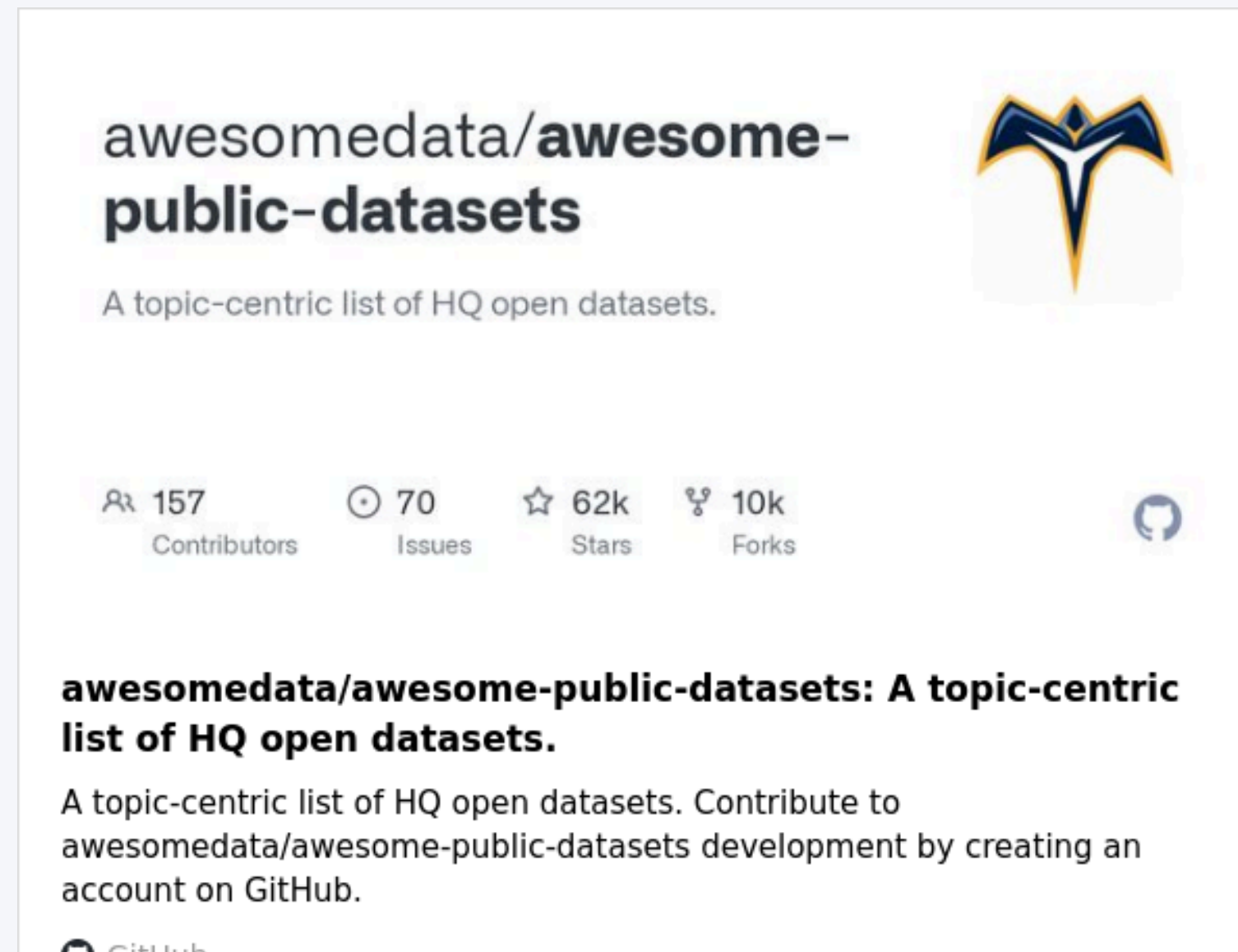
Billions of sensors have been collecting data for decades



Satellite and drone for monitor crop health and predict yields.

The Scale Challenge

- With so much driven by data, it's important that data scientists work **responsibly** and for the greater good



The Scale Challenge

Modern data science faces increasing challenges with data volume:

- Global data creation projected to exceed **180 zettabytes** by coming years.
- Single organizations routinely collect terabytes of data daily.
- Traditional tools break down with large-scale datasets.
- Complex computations become resource-intensive.

This course focuses on tools and techniques to overcome these scaling challenges.

Scaling Dimensions in Data Science

- **Vertical Scaling (Scale Up): Increasing resources on a single machine**
 - More RAM, faster CPUs, specialized hardware (GPUs)
- **Horizontal Scaling (Scale Out): Distributing across multiple machines**
 - Cluster computing, distributed processing frameworks
- **Algorithm Efficiency: Optimizing computational approaches**
 - Better algorithms, approximation methods, dimensionality reduction
- **Data Architecture: Smart data organization and storage**
 - Databases, data lakes, caching strategies

Introduction to Data Science

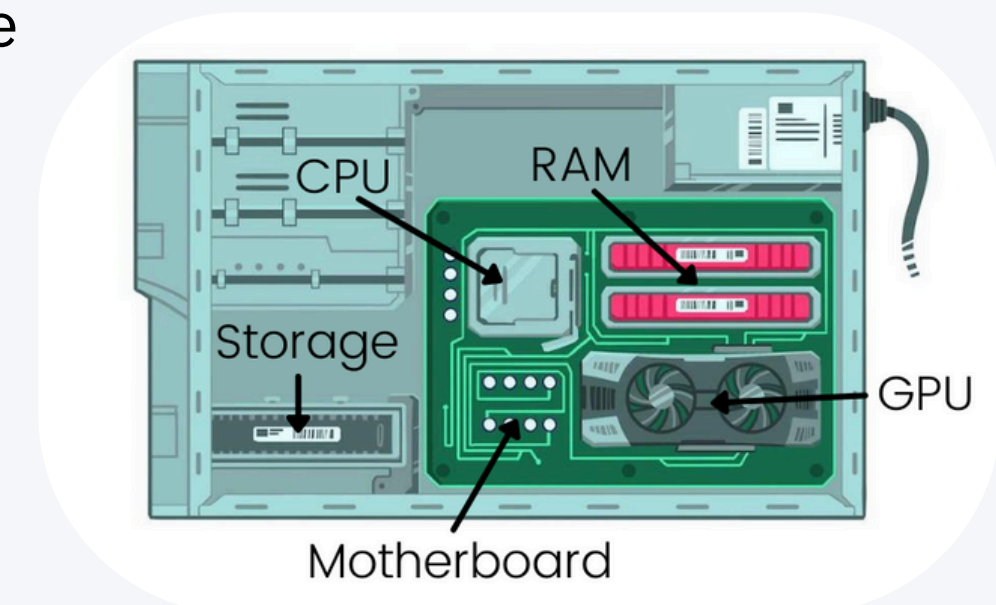
Important Terms in Scaling

CPU (Central Processing Unit):

- The "**brain**" of your computer
- Handles most of the general calculations and tasks
- Really good at doing a few complex tasks one after another
- Like a small team of very smart workers who can handle any task but work on one thing at a time
- Example: Running your operating system, opening applications, browsing the internet

GPU (Graphics Processing Unit):

- Originally designed for creating images and videos
- Excellent at doing many simple calculations all at once
- Like a large team of specialized workers who can do the same task simultaneously on different data
- Perfect for data science because analyzing data often involves repeating the same calculation on many data points
- Example: Rendering video games, training AI models, processing large datasets



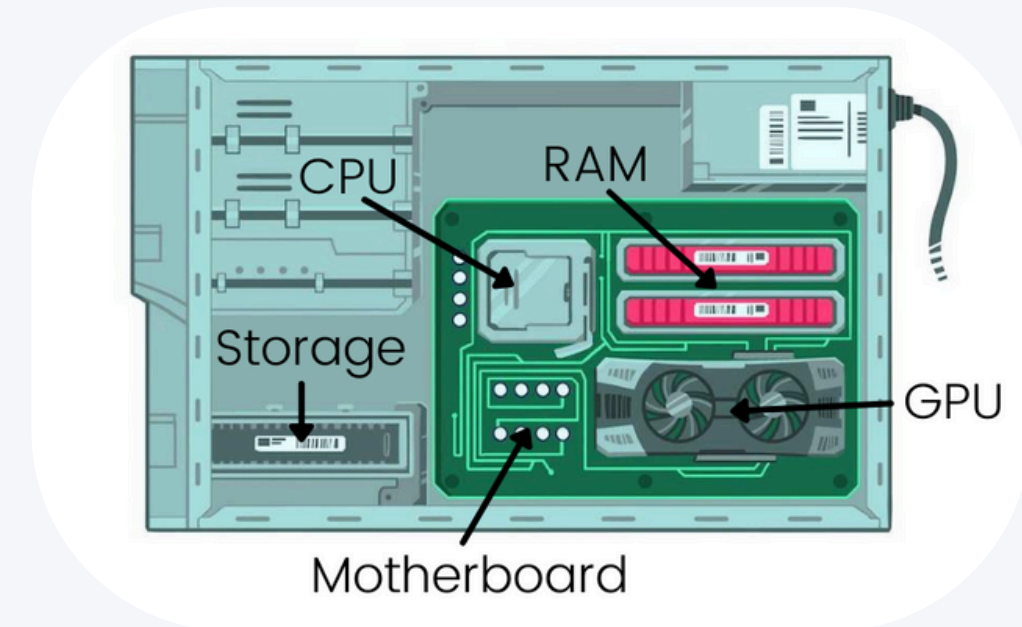
Introduction to Data Science



Important Terms in Scaling

RAM (Random Access Memory)

- The "workspace" or "short-term memory" of your computer
- Temporarily stores data that the CPU and GPU are actively using
- Allows for quick access to information without retrieving it from storage
- Like a desk where you keep documents you're currently working on
- Example: Holding the application data while you're using it, storing the data you're analyzing until the computer is turned off or the task is complete



In data science:

- **CPU** handles the overall program flow and complex operations
- **GPU** accelerates calculations when working with large datasets
- **RAM** determines how much data you can work with at once before needing to access storage

When you run out of RAM, everything slows down dramatically because the computer must use much slower storage (like your hard drive) as a substitute.

Vertical Scaling (Scale Up):

Increasing resources on a single machine

Hardware Upgrades:

RAM Expansion

Upgrading from 16GB to 64GB or 128GB RAM
Allows processing larger datasets entirely in memory

GPU Acceleration

Adding specialized NVIDIA Tesla or AMD Instinct GPUs
Upgrading from consumer GPUs to data center GPUs
with more VRAM

CPU Improvements

Adding more CPU cores (e.g., upgrading from a 4-core to a 32-core processor)
Using higher clock speed processors

Storage Enhancements

Switching from HDD to SSD or NVMe storage
Using RAID configurations for parallel read/write operations

Vertical Scaling (Scale Up):

Increasing resources on a single machine

Software Optimizations for Vertical Scaling

Multi-threading

Optimizing code to use all available CPU cores

GPU-Accelerated Libraries

Using CuPy instead of NumPy for array operations
Using cuDF instead of pandas for DataFrame operations

Memory-Efficient Algorithms

Using algorithms designed for minimal memory footprint

Vertical Scaling (Scale Up):

Increasing resources on a single machine

Limitations of Vertical Scaling:

- Physical constraints (maximum RAM supported by motherboard)
- Hardware limits (maximum available configurations)

Vertical scaling is often the simplest approach but eventually hits physical and economic limits, which is when horizontal scaling becomes necessary.

Horizontal Scaling (Scale Out):

Distributing across multiple machines

Computing Clusters

Hadoop Clusters

Setting up a network of commodity servers running HDFS and MapReduce

Example: Processing petabytes of log files using 100+ connected machines

Each node handles a portion of the data, enabling parallel processing

Spark Clusters

Deploying Apache Spark across multiple machines

Example: Running distributed machine learning with MLlib across a 20-node cluster

Allows in-memory distributed processing with fault tolerance

Dask Clusters

Scaling Python workflows across multiple machines

Example: Distributing pandas operations over a cluster to process DataFrames larger than a single machine's memory

Maintains familiar Python APIs while distributing computation

Horizontal Scaling (Scale Out):

Distributing across multiple machines

Cloud-Based Solutions

AWS EMR (Elastic MapReduce)

Dynamically scaling clusters based on workload
Example: Spinning up 500 nodes for an intensive overnight ETL job, then scaling down to save costs
Pay only for resources used during peak processing

Google Dataproc/BigQuery

Serverless distributed computing
Example: Analyzing billions of rows without managing infrastructure
Scales automatically based on query complexity

Horizontal Scaling (Scale Out): Distributing across multiple machines

Database Scaling

Sharding

Splitting databases across multiple servers by partitioning keys

Example: Distributing customer data geographically, with North American customers on one server cluster, European on another

Each machine handles queries for its specific data partition

Replication

Creating copies of data across multiple nodes
Example: Maintaining read-replicas to handle high-volume analytical queries without impacting transaction processing

Improves read performance while providing redundancy

Horizontal Scaling (Scale Out):

Distributing across multiple machines

Practical Implementations

Data-Parallel Processing

Breaking a dataset into chunks processed on separate machines

Example: Using PySpark to distribute CSV parsing across 50 nodes, each handling 2% of the total records
Results are combined after parallel processing

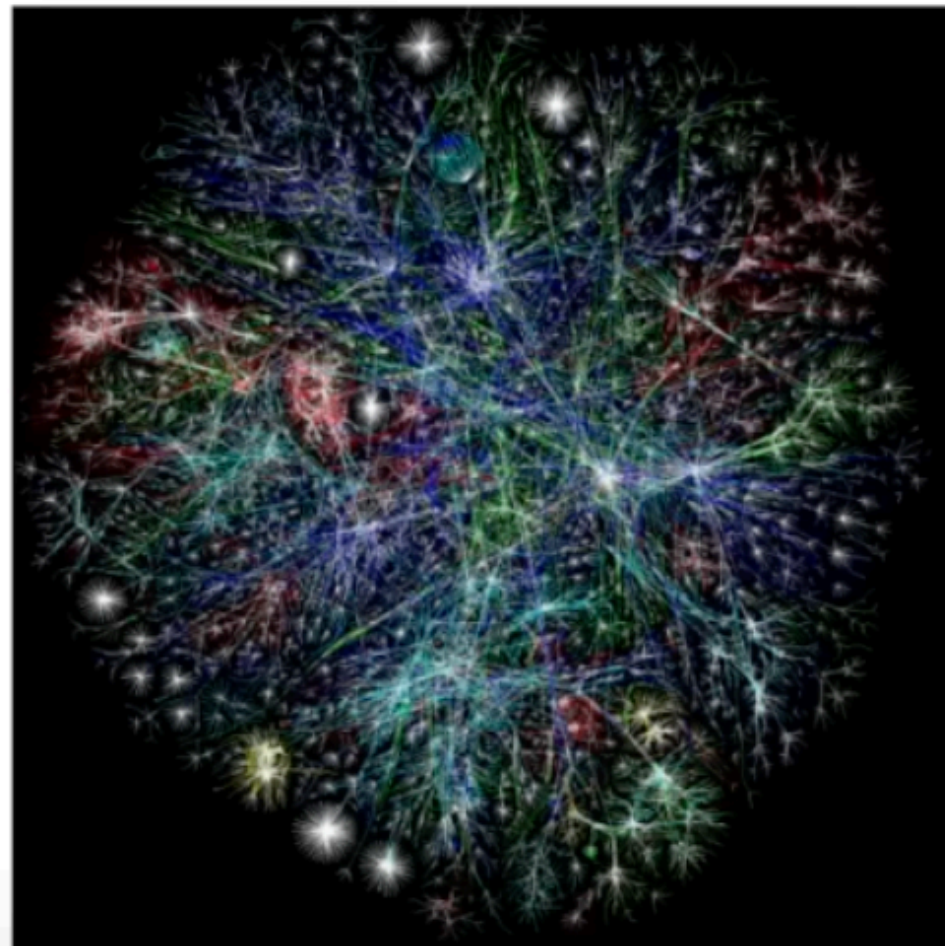
Model-Parallel Training

Distributing large machine learning models across multiple GPUs/machines

Example: Training massive language models where model parameters don't fit in a single GPU's memory
Different layers or components of the model run on different hardware

Introduction to Data Science

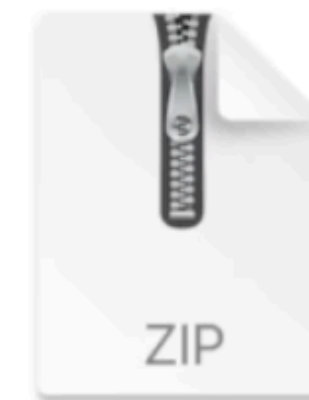
What About Training LLMs ?!



Chunk of the internet,
~10TB of text



6,000 GPUs for 12 days, ~\$2M
~1e24 FLOPS



parameters.zip

~140GB file

Why Python for Data Science?

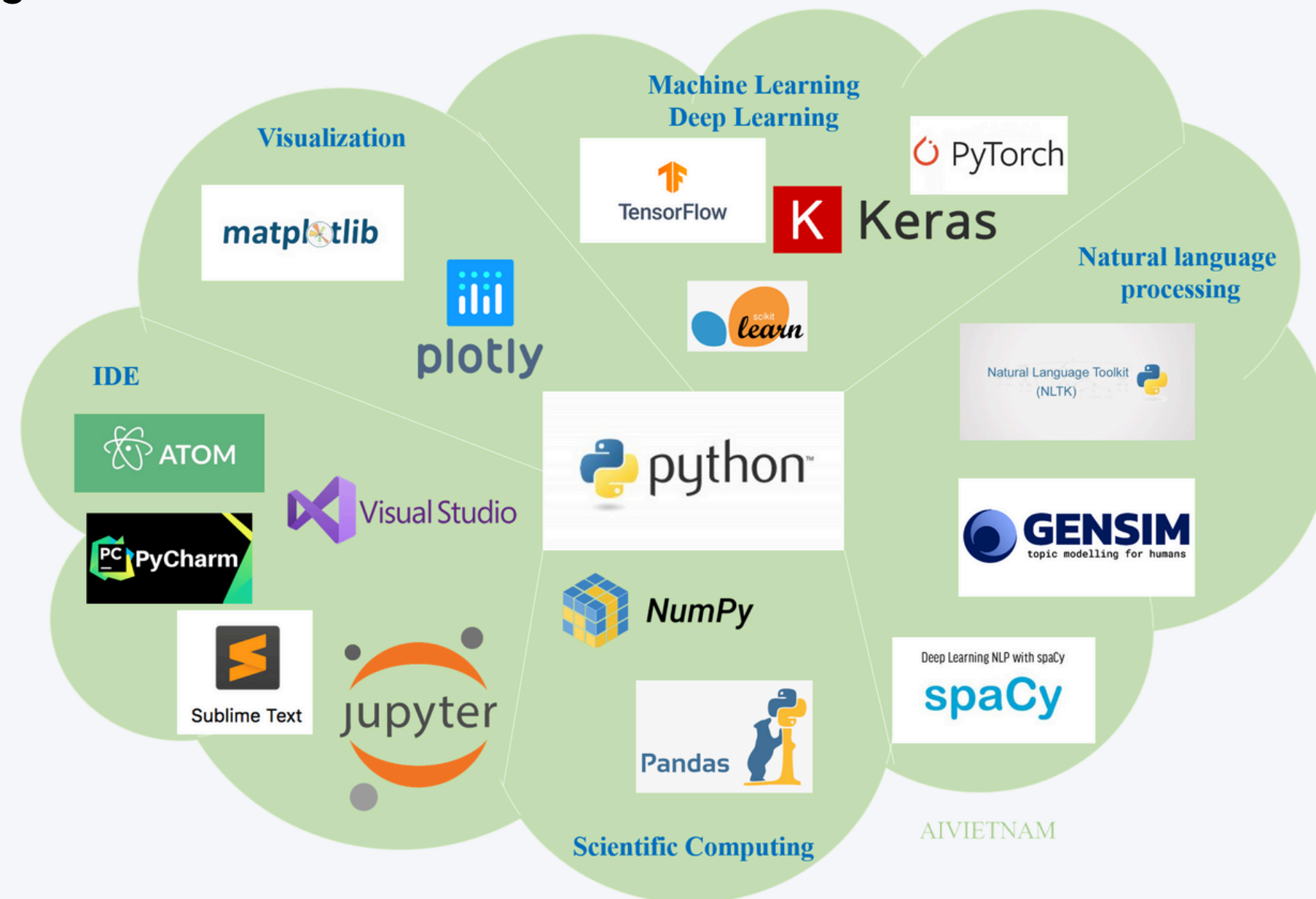
Python has become the de facto language for data science due to:

- **Readability and simplicity:** Easy to learn and use
- **Extensive ecosystem:** Rich libraries for data analysis and ML
- **Community support:** Large, active community and resources
- **Versatility:** Used across the entire data pipeline
- **Industry adoption:** Widely used in production environments
- **Integration capabilities:** Works well with other languages/systems

Introduction to Data Science

Python Ecosystem for Data Science

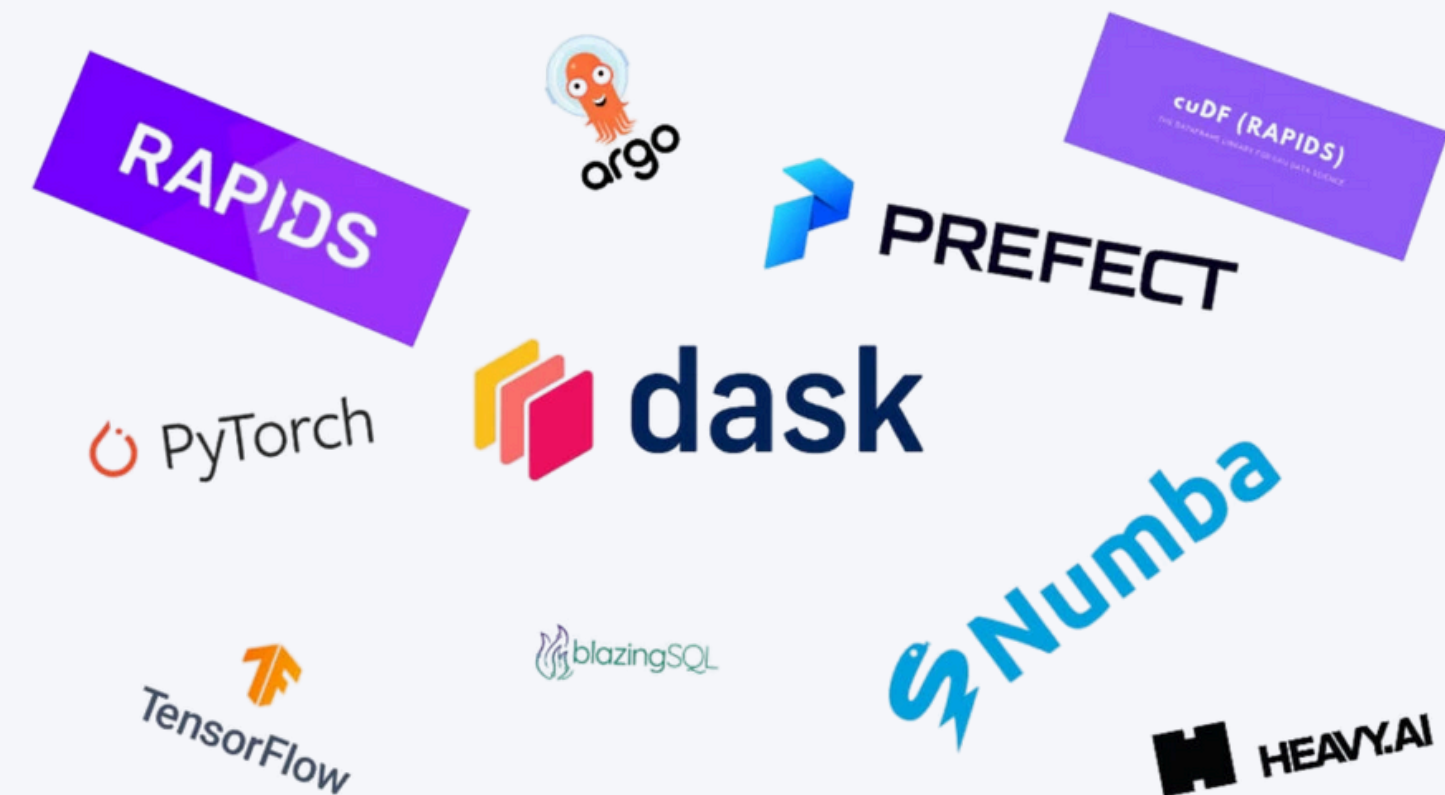
- **NumPy**: Numerical computing with multi-dimensional arrays
- **Pandas**: Data manipulation and analysis
- **Matplotlib/Seaborn**: Data visualization
- **Scikit-learn**: Machine learning algorithms
- **SciPy**: Scientific computing and technical computing
- **Statsmodels**: Statistical models and tests



Introduction to Data Science

Scaling Python: Core Technologies

- **NumPy**: Efficient numerical operations with vectorization.
- **Pandas**: Data manipulation and aggregation with optimized C backends
- **Dask**: Parallel computing library that scales Python
- **PySpark**: Python API for Apache Spark distributed computing
- **Ray**: Distributed computing framework for scaling Python
- **RAPIDS**: GPU-accelerated data science (CuPy, cuDF)
- **Numba**: JIT compiler for numeric Python functions





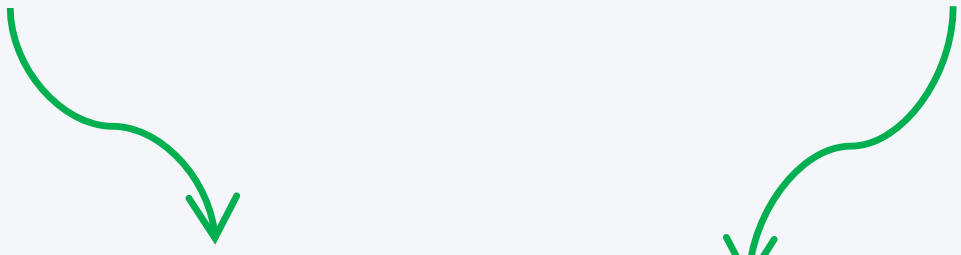
Libraries -Numpy

A popular math library in Python for Machine Learning is '**numpy**'.

Keyword to import a library

Keyword to refer to library by an alias (shortcut) name

`import numpy as np`



Numpy.org : NumPy is the fundamental package for scientific computing with Python.

- a powerful N-dimensional array object
- sophisticated (broadcasting) functions
- tools for integrating C/C++ and Fortran code
- useful linear algebra, Fourier transform, and random number capabilities



Libraries -Numpy

Numpy's main object is a multi-dimensional array.

Creating a NumpyArray as a Vector:

Value is: array([1, 2, 3])

Numpyfunction to create a numpyarray

data = np.array([1, 2, 3])

Creating a NumpyArray as a **Matrix**:

Outer Dimension

Inner Dimension (rows)

data = np.array([[1, 2, 3], [4, 5, 6], [7, 8, 9]])

Value is: array([1, 2, 3],
[4, 5, 6], [7, 8, 9])

Libraries -Numpy

Creating an **array of Zeros**:

Value is: array([0, 0, 0],
[0, 0, 0])

Numpyfunction to create an array of zeros

data type (default is float)

```
data = np.zeros( ( 2, 3 ), dtype=np.int )
```

rows

columns

Creating an **array of Ones**:

Value is: array([1, 1, 1],
[1, 1, 1])

Numpyfunction to create an array of ones

```
data = np.ones( (2, 3), dtype=np.int )
```

And many more functions: size, ndim, reshape, arange, ...

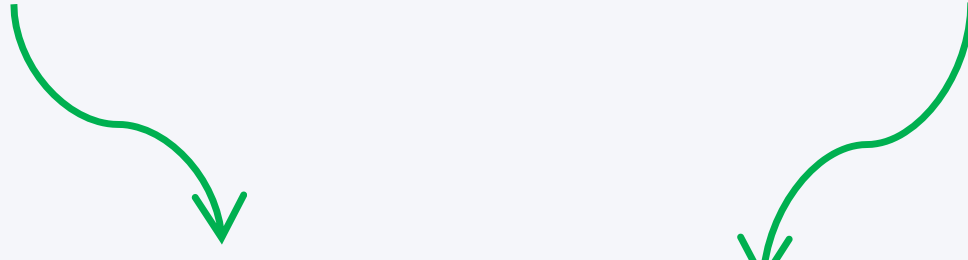
Libraries -Numpy

A popular library for importing and managing datasets in Python for Machine Learning is '**pandas**'.

Keyword to import a library

Keyword to refer to library by an alias (shortcut) name

`import pandas as pd`



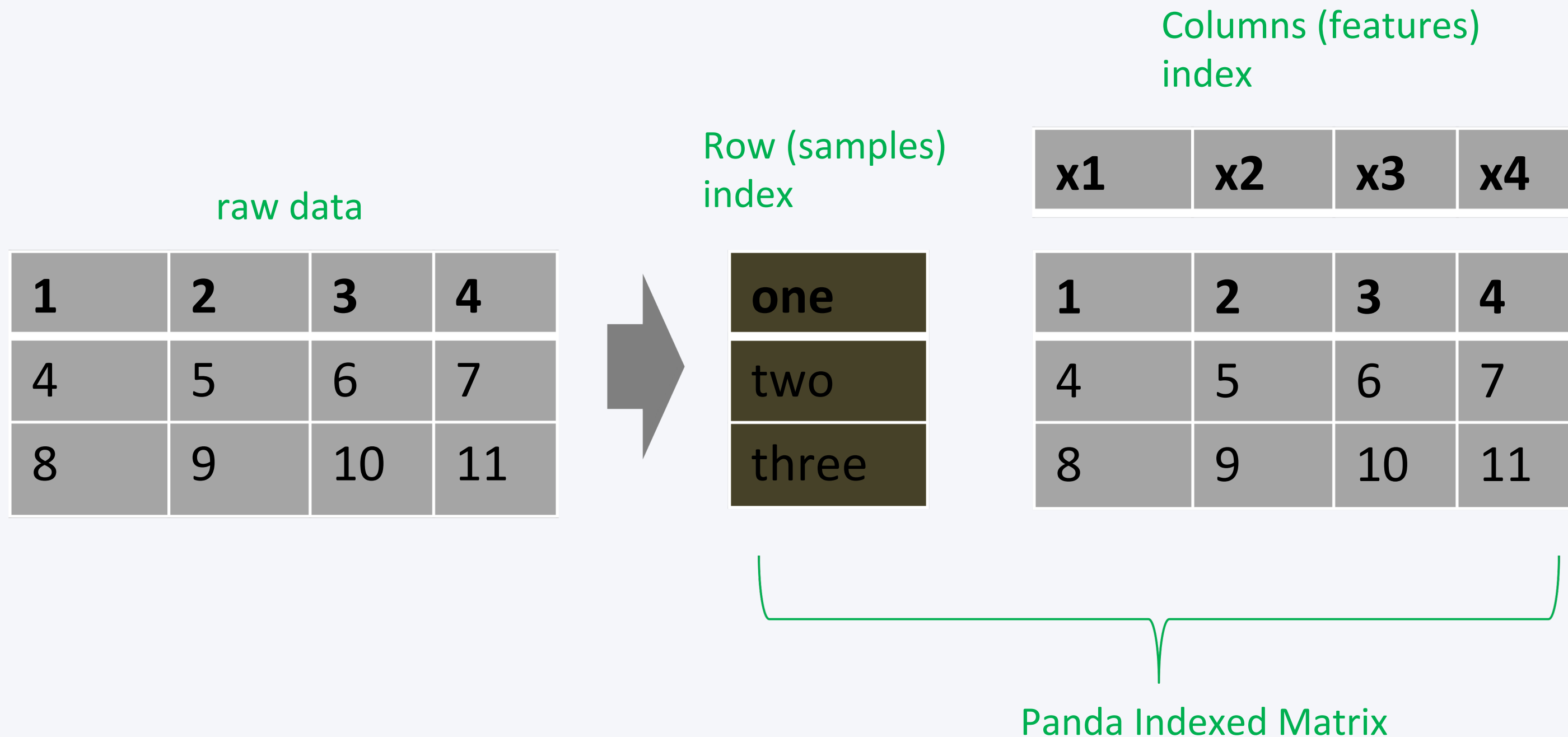
Used for:

- Data Analysis
- Data Manipulation
- Data Visualization

PyData.org : high-performance, easy-to-use data structures and data analysis tools for the Python programming language.

Pandas – Indexed Arrays

- Pandas are used to build indexed arrays (1D) and matrices(2D),
- where columns and rows are labeled (named) and can be accessed via the labels (names).



Pandas – Series and Data Frames

- Pandas Indexed Arrays are referred to as **Series**(1D) and **Data Frames**(2D).
- **Series** is a 1D labeled (indexed) array and can hold any data type, and mix of data types.

Series

Raw data

Column Index Labels

`s = pd.Series( data, index=[ 'x1', 'x2', 'x3', 'x4' ] )`

- **Data Frame** is a 2D labeled (indexed) matrix and can hold any data type, and mix of data types.

Data Frame

Row Index Labels

Column Index Labels

`df= pd.DataFrame( data, index=['one', 'two'], columns=[ 'x1', 'x2', 'x3', 'x4' ] )`

Pandas –Selecting

- Selecting One Column

Selects column labeled x1 for all rows

`x1 = df[ 'x1' ]`



1
4
8

- Selecting Multiple Columns

Selects columns labeled x1 and x3 for all rows

`x1 = df[ [ 'x1', 'x3' ] ]`



1	3
4	6
8	10

Selects columns labeled x1 through x3 for all rows

`x1 = df.ix[ :, 'x1':'x3' ]`

rows (all)

columns

Slicing function



1	2	3
4	5	6
8	9	10

Note: `df['x1':'x3']` this python syntax does not work!

And many more functions: merge, concat, stack, ...



Libraries -Matplotlib

A popular library for plotting and visualizing data in Python

Keyword to import a library

Keyword to refer to library by an alias (shortcut) name

`import matplotlib.pyplot as plt`

Used for:

- Histograms
- Plots
- Bar Charts
- Scatter Plots
- ...etc

matplotlib.org: Matplotlib is a Python 2D plotting library which produces publication quality figures in a variety of hardcopy formats and interactive environments across platforms.



Matplotlib - Plot

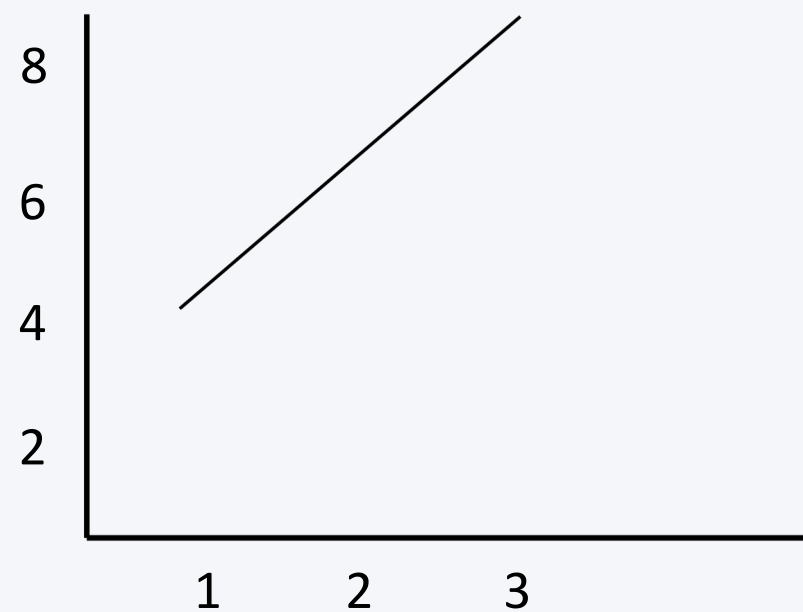
The function plot , **plots a 2D graph.**

Example:

Function to plot
X values to plot
Y values to plot

`plt.plot( x, y )`

`plt.plot( [ 1, 2, 3 ], [ 4, 6, 8 ] )` # Draws plot in the background
`plt.show()` # Displays the plot

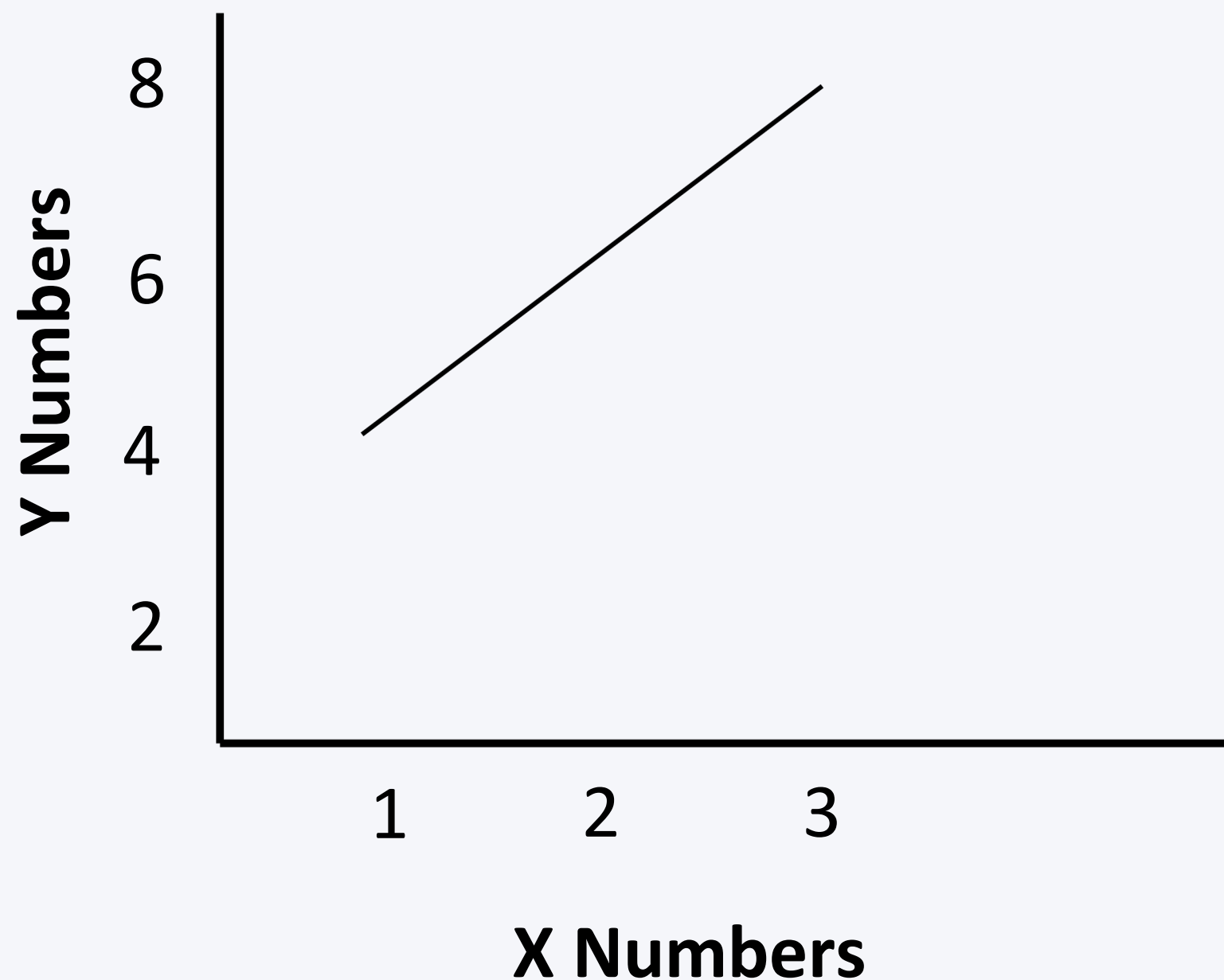




Matplotlib - Plot

Add Labels for X and Y Axis and Plot Title (caption)

My Plot of X and Y



```
plt.plot( [ 1, 2, 3 ], [ 4, 6, 8 ] )  
plt.xlabel( "X Numbers" )  
plt.ylabel( "Y Numbers" )  
plt.title( "My Plot of X and Y" )  
plt.show()
```

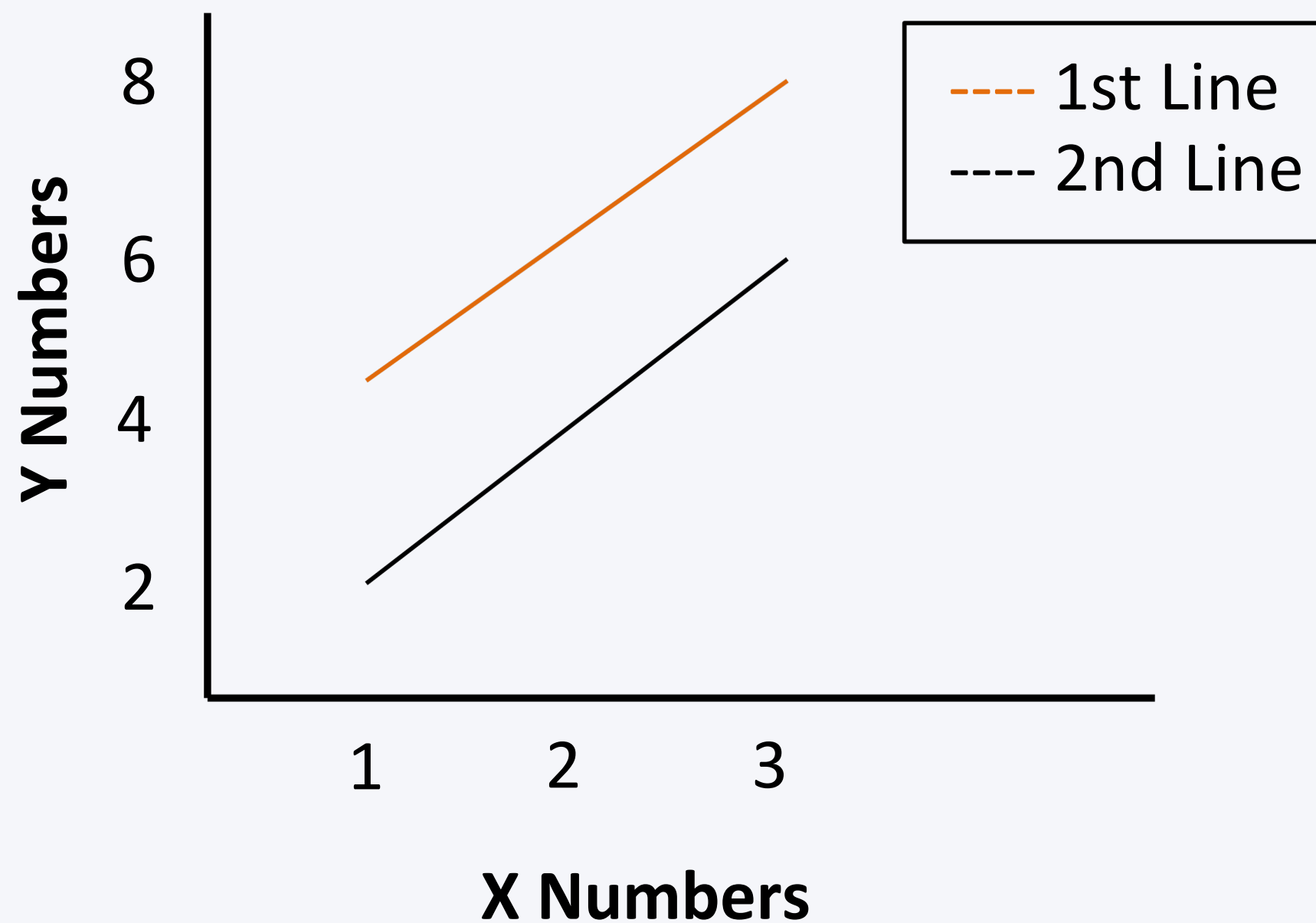
Label on the X-axis
Label on the Y-axis
Title for the Plot



Matplotlib–Multiple Plots and Legend

You can add multiple plots in a Graph

My Plot of X and Y



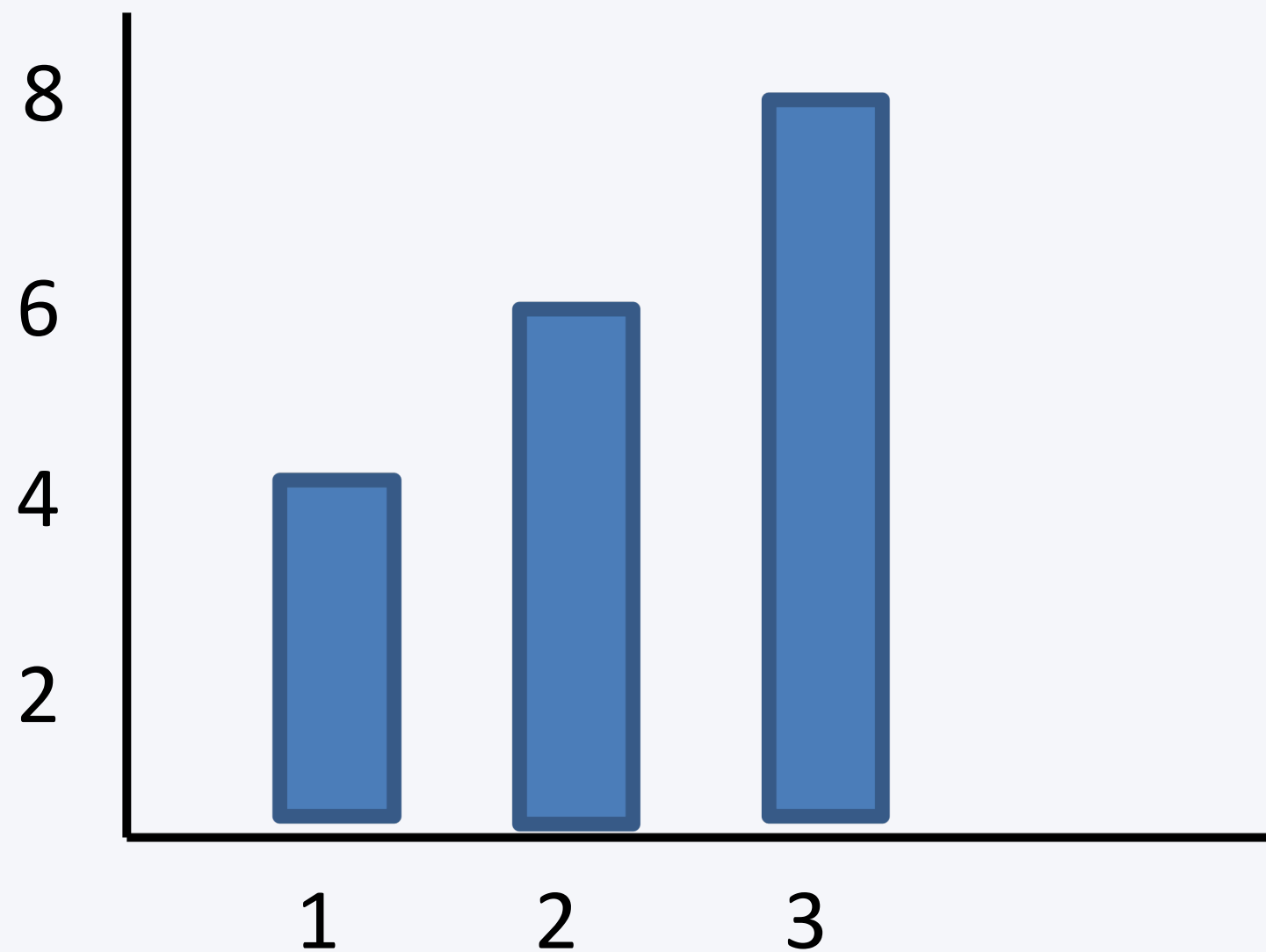
```
plt.plot( [ 1, 2, 3 ], [ 4, 6, 8 ], label=' 1st Line' )      # Plot for 1st Line
plt.plot( [ 1, 2, 3 ], [ 2, 4, 6 ], label='2nd Line' )      # Plot for 2nd Line
plt.xlabel( "X Numbers" )
plt.ylabel( "Y Numbers" )
plt.title( "My Plot of X and Y" )
plt.legend()
plt.show()
```

Show Legend for the plots



Matplotlib–Bar Chart

The function bar, **plots a bar graph**.



```
plt.plot( [ 1, 2, 3 ], [ 4, 6, 8 ] ) # Plot for 1stLine  
plt.bar() # Draw a bar chart  
plt.show()
```

And many more functions: hist, scatter, ...

Introduction to Data Science

Resources

